Java Project 1

Problem 1 :

Print all even and odd numbers from 1 to N

Code :

```
public class EvenOdd {
    public static void main(String[] args) {
        int n = 10;
        System.out.print("Even numbers: ");
        for (int i = 1; i <= n; i++) {
            if (i % 2 == 0) {
                System.out.print(i + " ");
            }
        }
        System.out.println();
        System.out.print("Odd numbers: ");
        for (int i = 1; i <= n; i++) {
            if (i % 2 != 0) {
                System.out.print(i + " ");
            }
        }
    }
}
```

Output :

Test case values 1 to 5

Even numbers : 2,4

Odd numbers : 1,3,5

Problem 2 :

Input a number and find the sum of its digits.
input:123 Output: 6

Code :

```
import java.util.Scanner;
public class SumOfDigits {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int sum = 0;
        while (n > 0) {
            int digit = n % 10;
            sum = sum + digit;
            n = n / 10;
        }
        System.out.println("Sum of digits = " + sum);
        sc.close();
    }
```

```
}
```

Output :

n = 123

output = 6

n = 4567

output = 22

Problem 3 :

Write a Java program to check whether a given number is a prime number or not.

Code :

```
import java.util.Scanner;
public class PrimeNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        boolean isPrime = true;
        if (n <= 1) {
            isPrime = false;
        }
        else {
```

```
            for (int i = 2; i < n; i++) {
                if (n % i == 0) {
                    isPrime = false;
                    break;
                }
            }
        }
        if (isPrime) {
            System.out.println(n + " is a Prime Number");
        }
        else {
            System.out.println(n + " is Not a Prime Number");
        }
        sc.close();
    }
}
```

Output :

n = 7

7 is a Prime Number

n = 12

12 is not a Prime Number

Problem 4 :

Input a number and reverse it. concept to use while loop
Example: 456 → 654

Code :

```
import java.util.Scanner;
public class ReverseNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter a number: ");
        int n = sc.nextInt();
        int reverse = 0;
        while (n > 0) {
            int digit = n % 10;
            reverse = reverse * 10 + digit;
            n = n / 10;
        }
        System.out.println("Reversed number = " + reverse);
        sc.close();
    }
}
```

Output :

n = 456
Reversed number = 654

n = 1245

reversed number = 5421

Problem 5 :

Using switch case, perform: Addition, Subtraction, Multiplication, Division switch case, if, break

Code :

```
import java.util.Scanner;
public class SimpleCalculator {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        double a = sc.nextDouble();
        System.out.print("Enter second number: ");
        double b = sc.nextDouble();
        System.out.print("Enter operator (+, -, *, /): ");
        char operator = sc.next().charAt(0);
        double result;
        switch (operator) {
            case '+':
                result = a + b;
                System.out.println("Addition = " + result);
                break;
            case '-':
```

```
            result = a - b;
            System.out.println("Subtraction = " + result);
            break;
        case '*':
            result = a * b;
            System.out.println("Multiplication = " + result);
            break;
        case '/':
            if (b != 0) {
                result = a / b;
                System.out.println("Division = " + result);
            } else {
                System.out.println("Error: Cannot divide by zero");
            }
            break;
        default:
            System.out.println("Invalid operator");
    }
    sc.close();
  }
}
```

Output :

First number = 20

Second number = 10

Operator = +

Addition = 30.0

a = 20

b = 0

operator = /

Error: Cannot divide by zero

Problem 6 :

Create a BankAccount class:

- Variables: accountNumber, balance
- Methods:
    - deposit(amount)
    - withdraw(amount)
    - checkBalance()

concepts to use Class & Object, Methods

Code :

```
class BankAccount {
    long accountNumber;
    double balance;
    BankAccount(long accountNumber, double balance) {
        this.accountNumber = accountNumber;
        this.balance = balance;
```

```
    }
    void deposit(double amount) {
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Deposited: " + amount);
        } else {
            System.out.println("Invalid deposit amount");
        }
    }
    void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid withdrawal amount");
        } else if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Withdrawn: " + amount);
        } else {
            System.out.println("Insufficient balance");
        }
    }
    void checkBalance() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Current Balance: " + balance);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        BankAccount account = new BankAccount(123456, 5000);
        account.checkBalance();
        account.deposit(2000);
        account.withdraw(1000);
        account.checkBalance();
    }
}
```

Output :

Account Number: 123456

Current Balance: 5000.0

Deposited: 2000.0

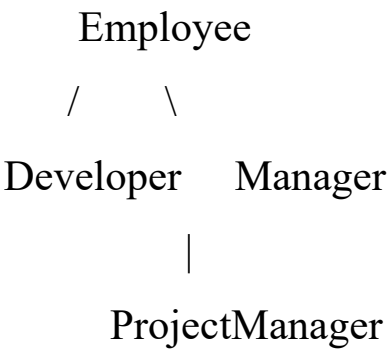
Withdrawn: 1000.0

Account Number: 123456

Current Balance: 6000.0

Problem 7:

Employee Inheritance Hierarchy



Code :

```
import java.util.ArrayList;
import java.util.List;
// Parent class
class Employee {
    int id;
    String name;
    double salary;
    Employee(int id, String name, double salary) {
        this.id = id;
        this.name = name;
        this.salary = salary;
    }
    void display() {
        System.out.println("Employee ID: " + id);
        System.out.println("Name: " + name);
```

```
        System.out.println("Salary: " + salary);
    }
}
// Child class 1
class Developer extends Employee {
    String programmingLanguage;
    Developer(int id, String name, double salary,
              String programmingLanguage) {
        super(id, name, salary);
        this.programmingLanguage = programmingLanguage;
    }
    @Override
    void display() {
       System.out.println("\n--- Developer Details ---");
        super.display();
        System.out.println(
            "Programming Language: " + programmingLanguage
        );
    }
}
// Child class 2
class Manager extends Employee {
    int teamSize;
    Manager(int id, String name, double salary, int teamSize) {
```

```
        super(id, name, salary);
        this.teamSize = teamSize;
    }
    @Override
    void display() {
        System.out.println("\n--- Manager Details ---");
        super.display();
        System.out.println("Team Size: " + teamSize);
    }
}
// Child class 3
class ProjectManager extends Manager {
    String projectName;
    ProjectManager(int id, String name, double salary,
                   int teamSize, String projectName) {
        super(id, name, salary, teamSize);
        this.projectName = projectName;
    }
    @Override
    void display() {
        System.out.println("\n--- Project Manager Details ---");
        super.display();
        System.out.println("Project Name: " + projectName);
    }
}
```

```
// Main class
public class Main {
    public static void main(String[] args) {
        // Parent class reference storing child objects
        List<Employee> employees = new ArrayList<>();
        employees.add(
            new Developer(101, "Arun", 45000, "Java")
        );
        employees.add(
            new Manager(102, "Ravi", 65000, 8)
        );
        employees.add(
            new ProjectManager(
                103, "Kumar", 85000, 12, "PitchBid"
            )
        );
        // Display all employee details
        for (Employee emp : employees) {
            emp.display();
        }
    }
}
```

Output :

--- Developer Details ---

Employee ID: 101

Name: Arun

Salary: 45000.0

Programming Language: Java

--- Manager Details ---

Employee ID: 102

Name: Ravi

Salary: 65000.0

Team Size: 8

--- Project Manager Details ---

Employee ID: 103

Name: Kumar

Salary: 85000.0

Team Size: 12

Project Name: PitchBid

Problem 8 :

Multiple Exception Handling in Java.

Code :

```
import java.util.Scanner;
import java.util.InputMismatchException;
public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] result = new int[2];
        try {
            System.out.print("Enter first number: ");
            int a = sc.nextInt();
            System.out.print("Enter second number: ");
            int b = sc.nextInt();
            int division = a / b;
            System.out.println("Division result: " + division);
            int index = 0;
            // Store result in array
            result[index] = division;
            System.out.println("Result stored successfully!");
            System.out.println("Array value: " + result[index]);
        }
```

```
        catch (ArithmeticException e) {
            System.out.println("Error: Cannot divide by zero!");
        }
        catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Invalid array index!");
        }
        catch (InputMismatchException e) {
            System.out.println("Error: Please enter integers only!");
        }
        finally {
            System.out.println("Program execution completed.");
            sc.close();
        }
    }
}
```

Output :

```
Enter first number: 20
Enter second number: 5
Division result: 4
Enter array index (0 or 1): 1
```

Problem 9 :

Employee Sorting using Comparable and Comparator in Java

Code :

```
import java.util.*;
class Employee implements Comparable<Employee> {
    int id;
    String name;
    double salary;
    int age;
    Employee(int id, String name, double salary, int age) {
        this.id = id;
        this.name = name;
        this.salary = salary;
        this.age = age;
    }
    // Natural sorting order: Name
    @Override
    public int compareTo(Employee other) {
        return this.name.compareToIgnoreCase(other.name);
    }
    void display() {
        System.out.println(
            "ID: " + id +
```

```
            " | Name: " + name +
            " | Salary: " + salary +
            " | Age: " + age
        );
    }
}
public class Main {
    public static void main(String[] args) {
        List<Employee> employees = new ArrayList<>();
        // Adding employees
        employees.add(new Employee(101, "Kumar", 45000, 25));
        employees.add(new Employee(102, "Arun", 60000, 28));
        employees.add(new Employee(103, "Ravi", 35000, 23));
        employees.add(new Employee(104, "Bala", 75000, 30));
        // Display original list
        System.out.println("Original Employee List:");
        for (Employee emp : employees) {
            emp.display();
        }
        // 1. Sort by Name using Comparable
        Collections.sort(employees);
        System.out.println("\nSorted by Name:");
        for (Employee emp : employees) {
            emp.display();
        }
```

```
        // 2. Sort by Salary using Comparator
        employees.sort(
            (e1, e2) -> Double.compare(e1.salary, e2.salary)
        );
        System.out.println("\nSorted by Salary:");
        for (Employee emp : employees) {
            emp.display();
        }
        // 3. Sort by Age using Comparator
        employees.sort(
            (e1, e2) -> Integer.compare(e1.age, e2.age)
        );
        System.out.println("\nSorted by Age:");
        for (Employee emp : employees) {
            emp.display();
        }
    }
}
```

Output :

Original Employee List:

ID: 101 | Name: Kumar | Salary: 45000.0 | Age: 25

ID: 102 | Name: Arun | Salary: 60000.0 | Age: 28

ID: 103 | Name: Ravi | Salary: 35000.0 | Age: 23

ID: 104 | Name: Bala | Salary: 75000.0 | Age: 30

Sorted by Name:

ID: 102 | Name: Arun | Salary: 60000.0 | Age: 28

ID: 104 | Name: Bala | Salary: 75000.0 | Age: 30

ID: 101 | Name: Kumar | Salary: 45000.0 | Age: 25

ID: 103 | Name: Ravi | Salary: 35000.0 | Age: 23

Sorted by Salary:

ID: 103 | Name: Ravi | Salary: 35000.0 | Age: 23

ID: 101 | Name: Kumar | Salary: 45000.0 | Age: 25

ID: 102 | Name: Arun | Salary: 60000.0 | Age: 28

ID: 104 | Name: Bala | Salary: 75000.0 | Age: 30

Sorted by Age:

ID: 103 | Name: Ravi | Salary: 35000.0 | Age: 23

ID: 101 | Name: Kumar | Salary: 45000.0 | Age: 25

ID: 102 | Name: Arun | Salary: 60000.0 | Age: 28

ID: 104 | Name: Bala | Salary: 75000.0 | Age: 30

Problem 10 :

Polymorphism using Animal, Dog, and Cat in Java.

Code :

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog says: Woof Woof!");
    }
}
class Cat extends Animal {
    @Override
    void sound() {
        System.out.println("Cat says: Meow Meow!");
    }
}
public class Main {
```

```
    public static void main(String[] args) {
        Animal animal1 = new Dog();
        Animal animal2 = new Cat();
        animal1.sound();
        animal2.sound();
    }
}
```

Output :

Dog says: Woof Woof!

Cat says: Meow Meow!

Problem 11 :

Finding Duplicate Elements in an Array using HashSet in Java.

Code :

```
import java.util.HashSet;
public class Main {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 2, 4};
        HashSet<Integer> set = new HashSet<>();
        boolean foundDuplicate = false;
        for (int num : arr){
```

```
            if (!set.add(num)) {
                System.out.println(
                    "Duplicate element found: " + num
                );
                foundDuplicate = true;
                break;
            }
        }
        if (!foundDuplicate) {
            System.out.println("No duplicate elements found.");
        }
    }
}
```

Output :

Duplicate element found: 2

Problem 12 :

Store Student Marks and Find a Student Using HashMap in Java.

Code :

```
import java.util.HashMap;
public class Main {
    public static void main(String[] args) {
```

```
        // Create HashMap
        HashMap<String, Integer> students = new HashMap<>();
        students.put("Arun", 85);
        students.put("Ravi", 90);
        students.put("Kumar", 78);
        int marks = students.get("Ravi");
        System.out.println("Ravi's Marks: " + marks);
    }
}
```

Output :

Ravi's Marks: 90

Problem 13 :

Create Class A with getName() and getDisplay() methods and handle exceptions

Code :

```
class A {
    String name;
    A(String name) {
```

```
        this.name = name;
    }
    // Method 1: getName()
    String getName() {
        try {
            if (name == null || name.isBlank()) {
                throw new Exception("Name cannot be empty or null");
            }
            return name;
        } catch (Exception e) {
            System.out.println("Exception in getName(): " +
e.getMessage());
            return null;
        }
    }
    // Method 2: getDisplay()
    void getDisplay() {
        try {
            String personName = getName();
            if (personName == null) {
                throw new Exception("Cannot display because name is
unavailable");
            }
            System.out.println("Name: " + personName);
        } catch (Exception e) {
```

```
            System.out.println("Exception in getDisplay(): " +
e.getMessage());
        }
    }
}
public class Main {
    public static void main(String[] args) {
        // Object with a valid name
        A obj1 = new A("Kishore");
        obj1.getDisplay();
        System.out.println("----------------");
        // Object with an invalid name
        A obj2 = new A(null);
        obj2.getDisplay();
    }
}
```

Output :

```
Name: Kishore
---------------
Exception in getName(): Name cannot be empty or null
Exception in getDisplay(): Cannot display because name is
unavailable
```

Problem 14 :

Find the Latest Version

Code :

```
public class Main {
    public static void main(String[] args) {
        String v1 = "3.11.54.2";
        String v2 = "3.11.53.1";
        String[] arr1 = v1.split("\\.");
        String[] arr2 = v2.split("\\.");
        int length = Math.max(arr1.length, arr2.length);
        for (int i = 0; i < length; i++) {
            int num1 = (i < arr1.length)
                    ? Integer.parseInt(arr1[i]) : 0;
            int num2 = (i < arr2.length)
                    ? Integer.parseInt(arr2[i]) : 0;
            if (num1 > num2) {
                System.out.println("Latest Version: " + v1);
                return;
            }
            else if (num1 < num2) {
                System.out.println("Latest Version: " + v2);
                return;
```

```
            }
        }
        System.out.println("Both versions are equal");
    }
}
```

Output :

Latest Version: 3.11.54.2

Problem 15 :

SQL Problem Solving

- Update employee records.
- Delete employee records.
- Sort records using ORDER BY.
- Group records using GROUP BY.
- Calculate totals using SUM().
- Convert names to uppercase and lowercase.

ID	Name	Age	Salary	Department	City	Manager
101	Arun	25	30000	IT	Chennai	108
102	Ravi	32	55000	HR	Bangalore	108
103	Kumar	28	45000	IT	Coimbatore	110

104	Priya	35	65000	HR	Chennai	110
105	Divya	26	32000	Finance	Bangalore	108
106	Manoj	29	48000	IT	Coimbatore	110
107	Sneha	31	52000	Finance	Chennai	108
108	Suresh	40	75000	Management	Bangalore	NULL
109	Vijay	24	28000	IT	Chennai	110
110	Bala	45	80000	Management	Chennai	NULL
111	Karthik	33	58000	HR	Coimbatore	108
112	Meena	27	38000	Finance	Bangalore	110
113	Ajay	36	62000	IT	Chennai	108
114	Lakshmi	30	42000	HR	Coimbatore	110
115	Gokul	23	25000	Finance	Chennai	108

Query 1: Update the manager_id of employee 108

Code :

```
UPDATE employee  SET manager_id = 115
WHERE employee_id = 108;
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id
108	Suresh	40	75000	Management	Bangalore	115

Query 2: Update salary greater than 50000.

Code :

```
UPDATE employee
SET salary = salary * 1.10
WHERE salary > 50000;
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id
102	Ravi	32	60500.00	HR	Bangalore	108
104	Priya	35	71500.00	HR	Chennai	110
107	Sneha	31	57200.00	Finance	Chennai	108
108	Suresh	40	82500.00	Management	Bangalore	115
110	Bala	45	88000.00	Management	Chennai	NULL
111	Karthik	33	63800.00	HR	Coimbatore	108
113	Ajay	36	68200.00	IT	Chennai	108

Query 3: Delete employee 110:

Code :

```
DELETE FROM employee
WHERE employee_id = 110;
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id

Query 4: Delete all HR employees:

Code :

```
DELETE FROM employee
WHERE department = 'HR';
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id

Query 5: Delete employees whose salary is less than 35000

Code :

```
DELETE FROM employee
WHERE salary < 110;
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id

Query 6: Delete employees from Bangalore

Code :

```
DELETE FROM employee
WHERE city = 'Bangalore';
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id

Query 7: Delete employees older than 30

Code :

```
DELETE FROM employee
WHERE age > 30;
```

Output :

employee_id employee_name age salary department city manager_id

Query 8: Display employees ordered by age

Code :

```
SELECT *
FROM employee
ORDER BY age ASC;
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id
103	Kumar	28	45000.00	IT	Coimbatore	110
106	Manoj	29	48000.00	IT	Coimbatore	110

Query 9: Sort employees by name

Code :

```
SELECT *
FROM employee
ORDER BY employee_name ASC;
```

Output :

employee_id	employee_name	age	salary	department	city	manager_id
103	Kumar	28	45000.00	IT	Coimbatore	110
106	Manoj	29	48000.00	IT	Coimbatore	110

Query 10: Count employees city-wise

Code :

```
SELECT city, COUNT(*) AS total_employees
FROM employee
GROUP BY city;
```

Output :

city	total_employees
Coimbatore	2

Query 12: Find the total salary of all employees

Code :

```
SELECT SUM(salary) AS total_salary
FROM employee;
```

Output :

total_salary
93000.00

Query 13: Display employee names in uppercase

Code :

```
SELECT UPPER(employee_name) AS employee_name
FROM employee;
```

Output :

employee_name
KUMAR
MANOJ

Query 14: Display employee names in lowercase

Code :

```
SELECT LOWER(employee_name) AS employee_name
FROM employee;
```

Output :

employee_name
kumar
manoj